

LAB 13

NAME: Hajra Sarwar

REG NO: 2023-BSE-022

CLASS: BSE-5A

SUBJECT: CC

LAB TITLE: Terraform IAM Management with AWS

TASK:

Task 0: Lab Setup (Codespace & GH CLI)

This task ensures that all work is performed inside a GitHub Codespace using GitHub CLI, as required. It prepares the working environment before starting Terraform tasks.

Steps

1. Create a new GitHub repository named `CC_<YourName>_<YourRollNumber>/Lab13` using GH CLI.
2. Create a GitHub Codespace for the Lab13 repository.
3. List available codespaces to confirm creation.

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022 (main) $ gh codespace list
NAME              DISPLAY NAME  REPOSITORY      BRANCH  STATE      CREATED AT
fluffy-tribble-wrq756... hlofaam      hajrasarwar11/CC... main*    Available  about 6 days ago
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022 (main) $
```

4. Connect to the Codespace using SSH.

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022 (main) $ gh codespace ssh -c fluffy-tribble-wrq756x47vr25q6q
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-1030-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
```

5. Verify that the terminal session is running inside the Codespace environment.

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022 (main) $ echo $CODESPACES || grep codespace /proc/1/cgroup || hostname
true
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022 (main) $
```

Task 1: Create IAM Group and Output Details

In this task, an IAM group named **developers** is created using Terraform. Terraform outputs are used to display the group's name, ARN, and unique ID after creation.

Steps

1. Create the project directory `Lab13` and navigate into it.

```

● @hajrasarwar11 →/workspaces/CC-HajraSarwar-022 (main) $ cd lab_13/Task
○ @hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_13/Task (main) $

```

2. Create the `main.tf` file.

```

● @hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_13/Task (main) $ touch main.tf
● @hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_13/Task (main) $ ls
main.tf  README.md

```

3. Configure the AWS provider using shared credentials.
4. Define an IAM group resource named `developers`.

```

lab_13 > Task > main.tf > output "group_details"
1  provider "aws" {
2      shared_config_files    = ["~/.aws/config"]
3      shared_credentials_files = ["~/.aws/credentials"]
4  }
5
6  resource "aws_iam_group" "developers" {
7      name = "developers"

```

5. Add Terraform outputs to display group details.
6. Initialize Terraform using `terraform init`.

```

@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_13/Task (main) $ terraform init
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

```

7. Apply the configuration using `terraform apply`.

```

@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_13/Task (main) $ terraform apply -auto
-approve
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

group_details = {
  "group_arn" = "arn:aws:iam::941618064465:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPA5WPGPNRIWSOVQPNE5"
}

```

8. Display outputs using `terraform output`.

```

● @hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_13/Task (main) $ terraform output
group_details = {
  "group_arn" = "arn:aws:iam::941618064465:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPA5WPGPNRIWSOVQPNE5"
}

```

9. Verify the IAM group in the AWS Console.

User groups (1) Info

A user group is a collection of IAM users. Use groups to specify permissions for a collection of users.

< 1 >

☐	Group name	Users	Permissions	Creation time
☐	developers	0	Not defined	5 minutes ago

This task creates an IAM user named **loadbalancer** and assigns the user to the **developers** group using Terraform.

Steps

1. Update `main.tf` to define an IAM user resource.

```
lab_13 > Task > main.tf > output "user_details"
11  output "group_details" {
17  }
18
19  resource "aws_iam_user" "lb" {
20    name = "loadbalancer"
21    path = "/users/"
22    force_destroy = true
23    tags = {
24      DisplayName = "Load Balancer"
25    }
26  }
```

2. Enable `force_destroy` to allow clean deletion.
3. Create a user-to-group membership resource.
4. Add Terraform outputs for user details.
5. Apply the updated configuration.

```
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022/lab_13/Task (main) $ terraform apply -auto
_approve
"group_arn" = "arn:aws:iam::941618064465:group/groups/developers"
"group_name" = "developers"
"unique_id" = "AGPA5WPGPNRIWSOVQPNE5"
}
user_details = {
  "unique_id" = "AIDA5WPGPNRI5E5P3HF3E"
  "user_arn" = "arn:aws:iam::941618064465:user/users/loadbalancer"
  "user_name" = "loadbalancer"
}
```

6. Display Terraform outputs.

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_13/Task (main) $ terraform output
group_details = {
  "group_arn" = "arn:aws:iam::941618064465:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPA5WPGPNRIWSOVQPNE5"
}
user_details = {
  "unique_id" = "AIDA5WPGPNRI5E5P3HF3E"
  "user_arn" = "arn:aws:iam::941618064465:user/users/loadbalancer"
  "user_name" = "loadbalancer"
```

7. Verify the user and group membership in the AWS Console.

Users (2) [Info](#)

Delete

Create user

An IAM user is an identity with long-term credentials that is used to interact with AWS in an account.

<

1

>

☐	User name	Path	Group	Last activity	MFA	Password age
☐	Admin	/	0	✓ 13 minutes ago	-	✓ 9 days
☐	loadbalancer	/users/	1	-	-	-

User groups membership

Remove

Add user to groups

A user group is a collection of IAM users. Use groups to specify permissions for a collection of users. A user can be a member of up to 10 groups at a time.

☐	Group name	Attached policies
☐	developers	-

Task 3: Attach Policies to IAM Group

In this task, AWS managed policies are attached to the **developers** group to grant EC2 access and allow users to change their passwords.

Steps

1. Update `main.tf` to add group policy attachment resources.



```
lab_13 > Task > main.tf > resource "aws_iam_group_policy_attachment" "change_password"
35  output "user_details" {
41  }
42
43  resource "aws_iam_group_policy_attachment" "developer_ec2_fullaccess" {
44    group = aws_iam_group.developers.name
45    policy_arn = "arn:aws:iam::aws:policy/AmazonEC2FullAccess"
46  }
```

2. Attach AmazonEC2FullAccess policy.
3. Attach IAMUserChangePassword policy.
4. Apply the Terraform configuration.

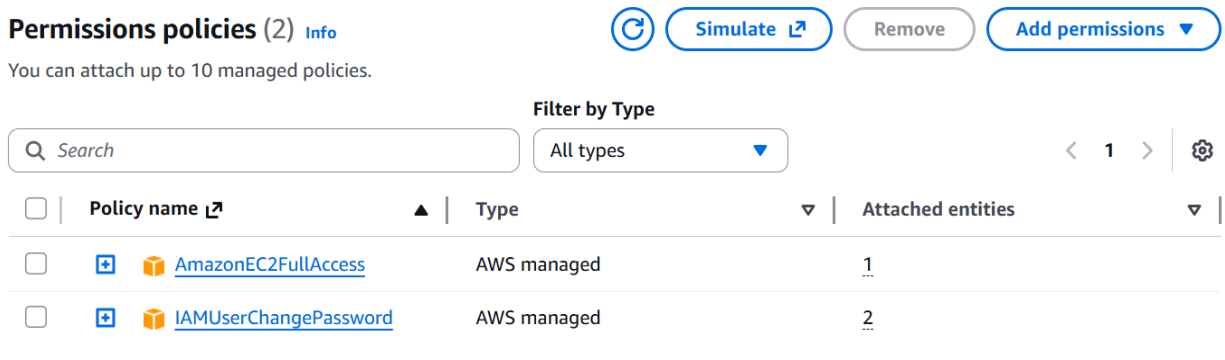


```
@hajrasarwar11 -> /workspaces/CC-HajraSarwar-022/lab_13/Task (main) $ terraform apply -auto
-approve
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

group_details = {
  "group_arn" = "arn:aws:iam::941618064465:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPA5WP6PNRTW5OVOPNE5"
```

5. Verify attached policies in the AWS Console.



Permissions policies (2) [Info](#) [Refresh](#) [Simulate](#) [Remove](#) [Add permissions](#)

You can attach up to 10 managed policies.

Filter by Type: All types

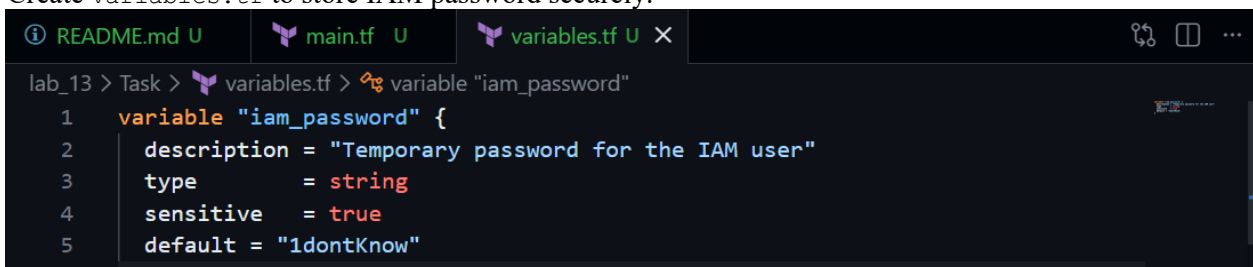
☐	Policy name ↗	Type	Attached entities
☐	AmazonEC2FullAccess	AWS managed	1
☐	IAMUserChangePassword	AWS managed	2

Task 4: Create Login Profile for IAM User

This task automates IAM console login profile creation using a Bash script and Terraform `null_resource` with `local-exec`.

Steps

1. Create `variables.tf` to store IAM password securely.



```
lab_13 > Task > variables.tf > variable "iam_password"
1  variable "iam_password" {
2    description = "Temporary password for the IAM user"
3    type        = string
4    sensitive   = true
5    default     = "1dontKnow"
```

2. Create `create-login-profile.sh` Bash script.

```

lab_13 > Task > $ create-login-profile.sh
1  #!/usr/bin/env bash
2  set -euo pipefail
3
4  USERNAME="$1"
5  PASSWORD="$2"
6
7  # Check if login profile already exists
8  if aws iam get-login-profile --user-name "$USERNAME" >/dev/null 2>&1; then

```

3. Make the script executable using `chmod`.

```

@hajrasarwar11 → /workspaces/CC-HajraSarwar-022/lab_13/Task (main) $ chmod +x create-login-profile.sh

```

4. Add a null resource with local-exec provisioner in `main.tf`.

```

lab_13 > Task > main.tf > resource "null_resource" "create_login_profile" > triggers
28  resource "null_resource" "create_login_profile" {
29      triggers = {
30          password_hash = sha256(var.iam_password)
31          user          = aws_iam_user.lb_name

```

5. Apply Terraform with a custom password variable.

```

@hajrasarwar11 → /workspaces/CC-HajraSarwar-022/lab_13/Task (main) $ terraform apply -auto-approve -var="iam_password=MySecurePass123!"
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

group_details = {
  "group_arn" = "arn:aws:iam::941618064465:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPA5WBG6BNPTMSOVOPNE5"

```

6. Verify login profile using AWS CLI.

```

@hajrasarwar11 → /workspaces/CC-HajraSarwar-022/lab_13/Task (main) $ aws iam get-login-profile --user-name loadbalancer
{
  "LoginProfile": {
    "UserName": "loadbalancer",
    "CreateDate": "2026-01-28T08:31:30+00:00",
    "PasswordResetRequired": true
  }
}

```

7. Test IAM user login in AWS Console.

Old Password

☐ Show Password

New Password

☐ Show Password

Confirm New Password

☐ Show Password

Matches

Confirm Password Change

[Sign in to a different account](#)

Europe (Stockholm)

hajra (9416-1806-4465)

loadbalancer

Reset to default layout

+ Add widgets

Task 5 — Generate Access Keys for IAM User

This task generates programmatic access keys for the IAM user and demonstrates how sensitive values are stored in Terraform state.

Steps

1. Add IAM access key resource to `main.tf`.

```
lab_13 > Task > main.tf > ...  
66 # Access key for IAM user  
67 resource "aws_iam_access_key" "lb_access_key" {  
68     user = aws_iam_user.lb.name  
69 }
```

2. Define outputs for access key ID and secret.
3. Apply Terraform configuration.

```
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022/lab_13/Task (main) $ terraform apply -auto  
-approve -var="iam_password=MySecurePass123!"  
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

4. Display outputs using `terraform output`.

```
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022/lab_13/Task (main) $ terraform output  
access_key_id = "AKIA5WPGPNRIYT2DNYFM"  
access_key_secret = <sensitive>  
group_details = {  
  "group_arn" = "arn:aws:iam::941618064465:group/groups/developers"  
  "group_name" = "developers"  
  "unique_id" = "AGPA5WPGPNRIWSOVQPNE5"  
}  
user_details = {  
  "unique_id" = "AIDA5WPGPNRI5E5P3HF3E"  
  "user_arn" = "arn:aws:iam::941618064465:user/users/loadbalancer"  
  "user_name" = "loadbalancer"
```

5. View secret value in `terraform.tfstate`.

```
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022/lab_13/Task (main) $ cat terraform.tfstate  
| grep -A 10 "access_key_secret"  
  "access_key_secret": {  
    "value": "09BbFIWKW7J5vRngwSzuIVZTiMkUXFtCMoBc9122",  
    "type": "string",  
    "sensitive": true  
  },  
  "group_details": {  
    "value": {  
      "group_arn": "arn:aws:iam::941618064465:group/groups/developers",  
      "group_name": "developers",  
      "unique_id": "AGPA5WPGPNRIWSOVQPNE5"
```

6. Verify access keys in AWS Console.

Access keys (1)

[Create access key](#)

Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time. [Learn more](#)

AKIA5WPGPNRIYT2DNYFM

Description

-

Last used

None

Last used region

N/A

Status

✓ Active

Created

6 minutes ago

Last used service

N/A

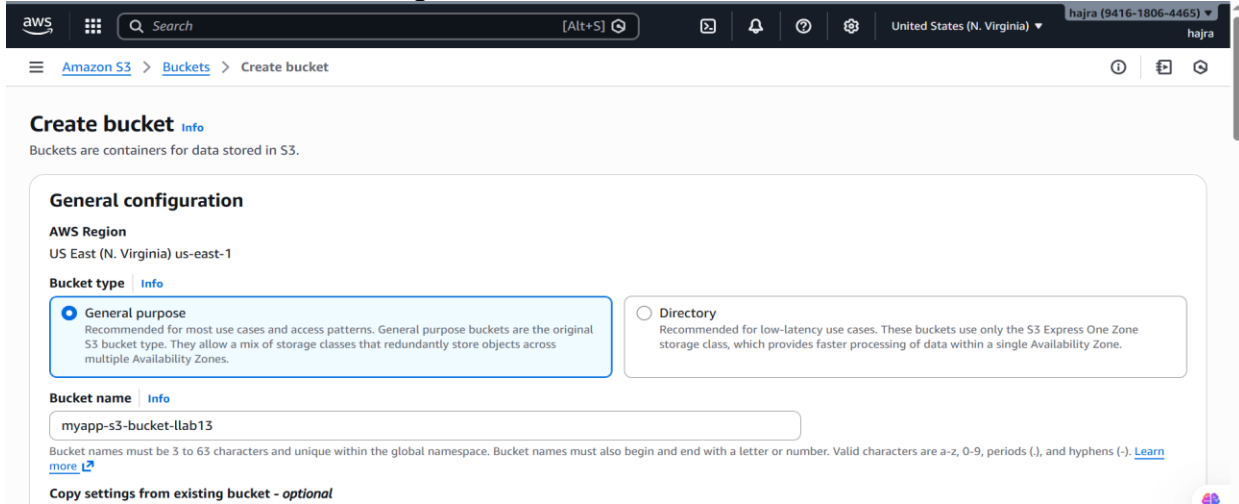
[Actions](#)

Task 6 — Implement Terraform Remote State with S3

This task configures Terraform to store its state remotely in an S3 bucket with versioning enabled.

Steps

1. Create an S3 bucket with versioning enabled.



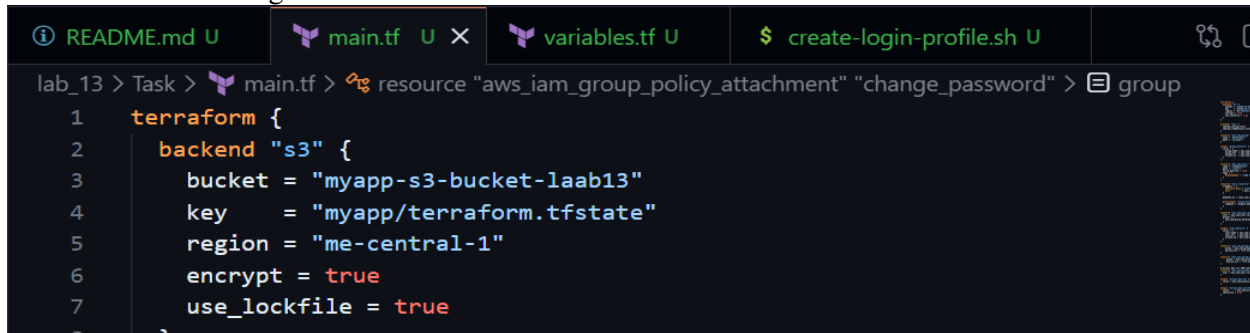
Bucket Versioning

Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to preserve, retrieve, and restore every version of every object stored in your Amazon S3 bucket. With versioning, you can easily recover from both unintended user actions and application failures. [Learn more](#)

Bucket Versioning

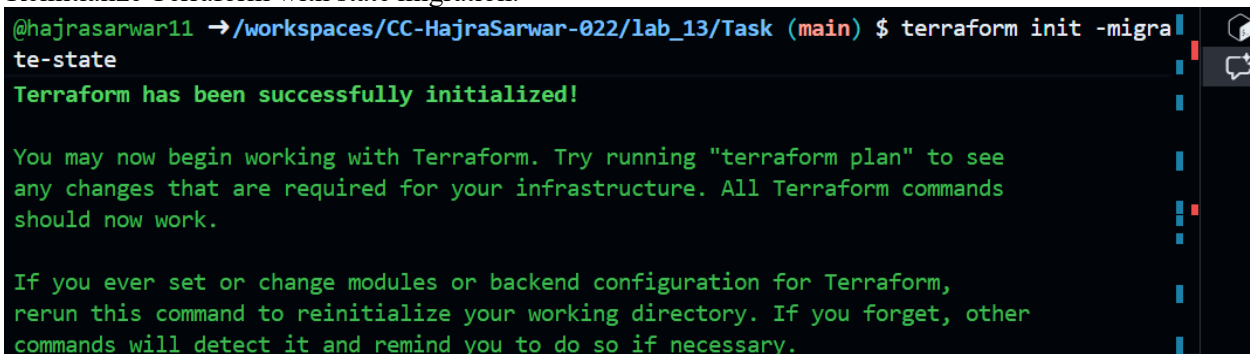
- ☐ Disable
- ☒ Enable

2. Add S3 backend configuration to `main.tf`.



```
lab_13 > Task > main.tf > resource "aws_iam_group_policy_attachment" "change_password" > group
1 terraform {
2   backend "s3" {
3     bucket = "myapp-s3-bucket-lab13"
4     key    = "myapp/terraform.tfstate"
5     region = "me-central-1"
6     encrypt = true
7     use_lockfile = true
8   }
9 }
```

3. Reinitialize Terraform with state migration.

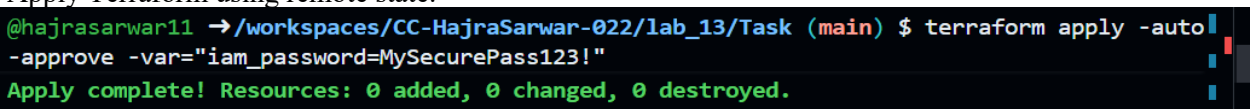


```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_13/Task (main) $ terraform init -migrate-state
Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

4. Apply Terraform using remote state.



```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_13/Task (main) $ terraform apply -auto-approve -var="iam_password=MySecurePass123!"
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
```

5. Verify state file in S3.

Amazon S3 > Buckets > myapp-s3-bucket-laab13 > myapp/

myapp/ Copy S3 URI

Objects (1) Copy S3 URI Copy URL Download Open Delete Actions Create folder Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix Show versions < 1 > Settings

☐	Name	Type	Last modified	Size	Storage class
☐	terraform.tfstate	tfstate	January 28, 2026, 14:02:36 (UTC+05:00)	6.7 KB	Standard

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_13/Task (main) $ ls -la terraform.tfstate*
-rw-rw-rw- 1 vscode vscode 0 Jan 28 09:02 terraform.tfstate
-rw-rw-rw- 1 vscode vscode 6882 Jan 28 09:02 terraform.tfstate.backup
```

6. Destroy resources and verify updated state.

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_13/Task (main) $ terraform destroy -auto-approve
aws_iam_group_policy_attachment.developer_ec2_fullaccess: Destruction complete after 1s
aws_iam_user_group_membership.lb_membership: Destruction complete after 1s
aws_iam_group.developers: Destroying... [id=developers]
aws_iam_access_key.lb_access_key: Destruction complete after 1s
aws_iam_user.lb: Destroying... [id=loadbalancer]
aws_iam_group.developers: Destruction complete after 1s
aws_iam_user.lb: Destruction complete after 3s

Destroy complete! Resources: 7 destroyed.
```

Amazon S3 > Buckets > myapp-s3-bucket-laab13 > myapp/

myapp/ Copy S3 URI

Objects (0) Copy S3 URI Copy URL Download Open Delete Actions Create folder Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix Show versions < 1 > Settings

☐	Name	Type	Last modified	Size	Storage class
No objects					

Task 7 — Create Multiple Users from CSV File

This task demonstrates scalable IAM user creation using a CSV file and Terraform `for_each`.

Steps

1. Create `locals.tf` to decode CSV data.

```
lab_13 > Task > locals.tf > locals
1 locals {
2   users = csvdecode(file("users.csv"))
3 }
```

2. Create `users.csv` with multiple usernames.

```
lab_13 > Task > users.csv
1 user_name
2 Michael
3 Dwight
4 Jim
5 Pam
6 Ryan
7 Andy
```


3. Update `main.tf` to create users dynamically.

```
lab_13 > Task > main.tf > resource "aws_iam_access_key" "users_access_keys" > user

30 # Create multiple IAM users from CSV
31 resource "aws_iam_user" "users" {
32     for_each = { for user in local.users : user.user_name => user }
33
34     name      = each.value.user_name
35     path      = "/users/"
36     force_destroy = true
```

4. Assign all users to developers group.

```
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022/lab_13/Task (main) $ terraform init
```

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

5. Generate login profiles and access keys.

6. Apply Terraform configuration.

A screenshot of the Visual Studio Code (VS Code) interface. The top bar shows several tabs: 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is selected and underlined), and 'PORTS'. To the right of these tabs are icons for a chat window, a terminal window, and a trash can. The 'TERMINAL' tab shows a green message: 'Apply complete! Resources: 107 added, 0 changed, 0 destroyed.'

7. Verify users, groups, and state file.

The screenshot displays the Visual Studio Code editor with a terminal window open. The terminal shows the output of a Terraform command executed in the directory `/workspaces/CC-HajraSarwar-022/lab_13/Task`. The output lists the details for four users defined in the configuration: Robert, Ryan, Stanley, and Toby. Each user entry includes their access key ID, ARN, and unique ID. Additionally, the output shows the details for a group named 'developers'.

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_13/Task (main) $ terraform output

"Robert" = {
  "access_key_id" = "AKIASWPGPNRI7QXIMF8D"
  "user_arn" = "arn:aws:iam:941618064465:user/users/Robert"
  "user_unique_id" = "AIDASWPGPNRI6WD7NIW4B"
}

"Ryan" = {
  "access_key_id" = "AKIASWPGPNRIVRSAIRU7"
  "user_arn" = "arn:aws:iam:941618064465:user/users/Ryan"
  "user_unique_id" = "AIDASWPGPNRI2C7VAJ3CC"
}

"Stanley" = {
  "access_key_id" = "AKIASWPGPNRIYC2XGYH2"
  "user_arn" = "arn:aws:iam:941618064465:user/users/Stanley"
  "user_unique_id" = "AIDASWPGPNRI6MAUCS45U"
}

"Toby" = {
  "access_key_id" = "AKIASWPGPNRI4V6ZFA72"
  "user_arn" = "arn:aws:iam:941618064465:user/users/Toby"
  "user_unique_id" = "AIDASWPGPNRIQZ5FJVTT"
}

group_details = {
  "group_arn" = "arn:aws:iam:941618064465:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPASWPGPNRI2DIGVZXS"
```

Terraform secrets:

```

@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_13/Task (main) $ cat terraform.tfstate
| grep -A 5 "all_access_key_secrets"
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_13/Task (main) $

```

The output for `all_access_key_secrets` is marked as sensitive in the Terraform configuration. As a result, the actual secret values are not displayed in the console output or when using `grep` on the state file. This is a security best practice in Terraform to prevent accidental exposure of sensitive information.

aws

Search

[Alt+S]

Global

hajra (9416-1806-4465)

hajra

IAM > Users

Identity and Access Management (IAM)

Search IAM

Dashboard

Access Management

User groups

Users

Roles

Policies

Identity providers

Users (27)

Info

An IAM user is an identity with long-term credentials that is used to interact with AWS in an account.

Search

< 1 2 >

☐	User name	Path	Group:	Last activity	MFA	Password age
☐	Admin	/	0	25 minutes ago	-	9 days
☐	Andy	/users/	1	-	-	26 minutes
☐	Angela	/users/	1	-	-	26 minutes
☐	Charles	/users/	1	-	-	26 minutes
☐	Clark	/users/	1	-	-	26 minutes

aws

Search

[Alt+S]

Global

hajra (9416-1806-4465)

hajra

IAM > User groups > developers

Identity and Access Management (IAM)

Search IAM

Dashboard

Access Management

User groups

Users

Roles

...

Users in this group (26)

Remove

Add users

An IAM user is an entity that you create in AWS to represent the person or application that uses it to interact with AWS.

Search

< 1 2 >

☐	User name	Groups	Last activity	Creation time
☐	Andy		None	28 minutes ago
☐	Angela		None	28 minutes ago
☐	Charles		None	28 minutes ago
☐	Clark		None	28 minutes ago

aws

Search

[Alt+S]

Global

hajra (9416-1806-4465)

hajra

IAM > Users > Angela

Identity and Access Management (IAM)

Search IAM

Dashboard

Access Management

User groups

Users

Roles

Access keys (1)

Create access key

Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time. [Learn more](#)

AKIA5WPGPNRI6SFILOUR

Description

-

Last used

None

Last used region

N/A

Status

Active

Created

29 minutes ago

Last used service

N/A

Actions

aws

Search

[Alt+S]

Middle East (UAE)

hajra (9416-1806-4465)

hajra

Amazon S3 > Buckets > myapp-s3-bucket-laab13 > myapp/ > terraform.tfstate

Amazon S3

Buckets

General purpose buckets

Directory buckets

Table buckets

Vector buckets

Access management and security

Access Points

Access Points for FSx

Access Grants

IAM Access Analyzer

Storage management and insights

Storage Lens

terraform.tfstate

Info

Copy S3 URI

Download

Open

Object actions

Properties

Permissions

Versions

Object overview

Owner

98399dcf824dcc012e5979aed346b3c6482a423cf1bd9234c2d7702f86559af9

AWS Region

Middle East (UAE) me-central-1

Last modified

January 28, 2026, 14:18:37 (UTC+05:00)

Size

95.0 KB

S3 URI

s3://myapp-s3-bucket-laab13/myapp/terraform.tfstate

Amazon Resource Name (ARN)

arn:aws:s3::myapp-s3-bucket-laab13/myapp/terraform.tfstate

Entity tag (Etag)

40ce8e94e29c3d15782a285fb6f91ef8

Object URL

https://myapp-s3-bucket-laab13.s3.me-central-1.amazonaws.com/myapp/terraform.tfstate

Cleanup — Destroy Resources & Verify State

This task ensures that all AWS resources are properly removed and no residual state remains.

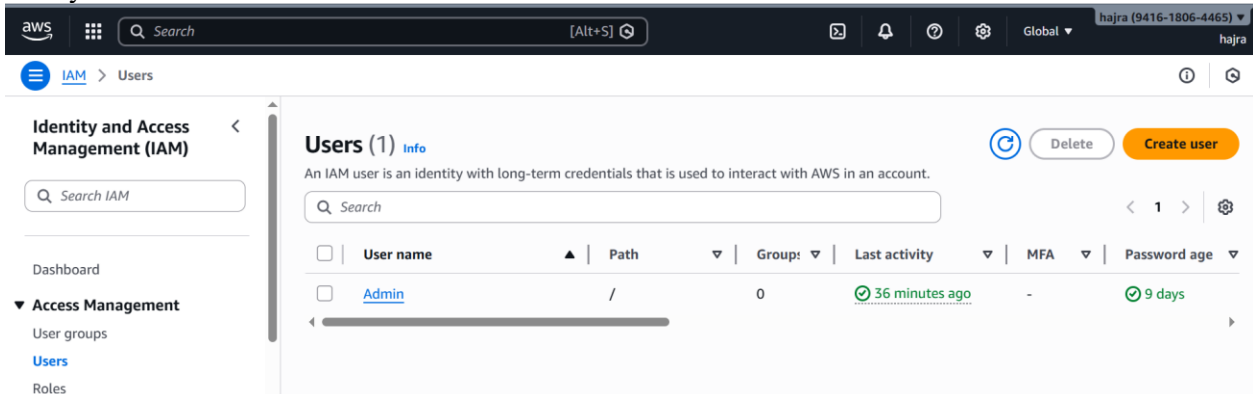
Steps

1. Destroy all Terraform-managed resources.

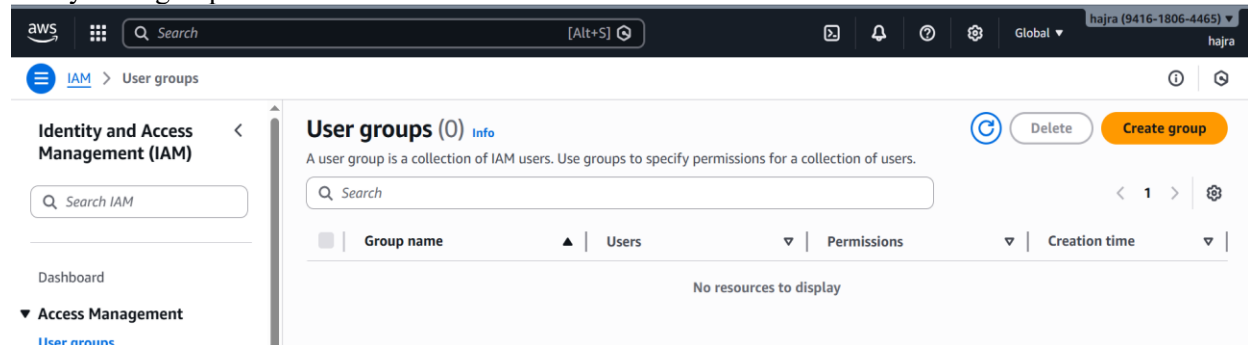
```
aws_iam_user.users["Dwight"]: Destruction complete after 3s
aws_iam_user.users["Phyllis"]: Destruction complete after 3s
aws_iam_user.users["Robert"]: Destruction complete after 3s
aws_iam_user.users["Jim"]: Destruction complete after 5s
aws_iam_user.users["David"]: Destruction complete after 3s
aws_iam_user.users["Peter"]: Destruction complete after 3s
aws_iam_user.users["Michael"]: Destruction complete after 3s
aws_iam_user.users["Meredith"]: Destruction complete after 5s
aws_iam_user.users["Kevin"]: Destruction complete after 8s

Destroy complete! Resources: 107 destroyed.
```

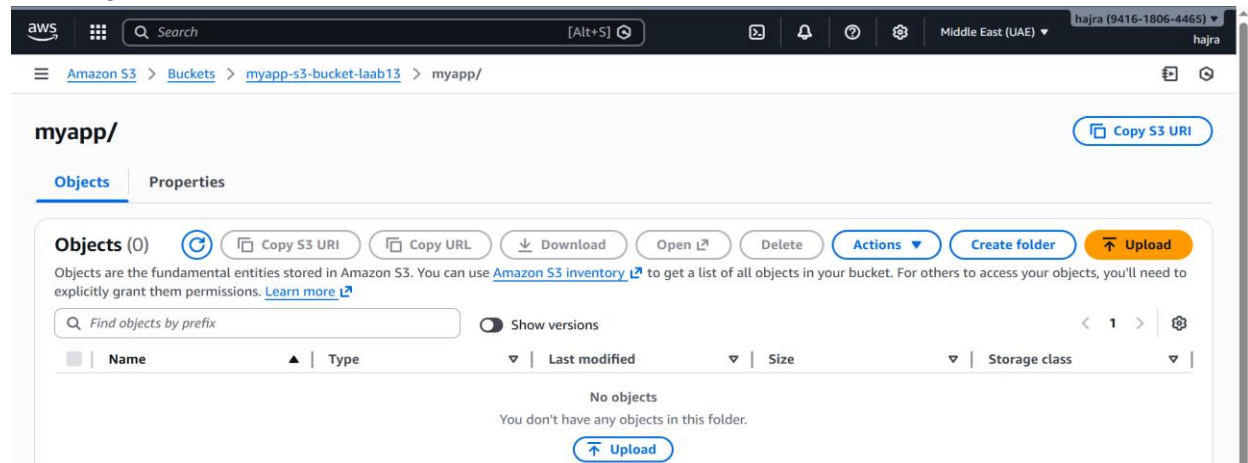
2. Verify IAM users deletion in AWS Console.



3. Verify IAM group deletion.



4. Check S3 state file.



5. List final project files.

```

@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_13/Task (main) $ ls -la
total 44
drwxrwxrwx+ 3 vscode vscode 4096 Jan 28 09:12 .
drwxrwxrwx+ 3 vscode vscode 4096 Jan 28 07:46 ..
-rwxrwxrwx 1 vscode vscode 422 Jan 28 08:26 create-login-profile.sh
-rw-rw-rw- 1 vscode vscode 49 Jan 28 09:11 locals.tf
-rw-rw-rw- 1 vscode vscode 2535 Jan 28 09:16 main.tf
-rw-rw-rw- 1 vscode vscode 0 Jan 28 07:46 README.md
drwxr-xr-x+ 3 vscode vscode 4096 Jan 28 09:02 .terraform
-rw-r--r-- 1 vscode vscode 2422 Jan 28 08:31 .terraform.lock.hcl
-rw-rw-rw- 1 vscode vscode 0 Jan 28 09:02 terraform.tfstate
-rw-rw-rw- 1 vscode vscode 6882 Jan 28 09:02 terraform.tfstate.backup
-rw-rw-rw- 1 vscode vscode 166 Jan 28 09:13 users.csv
-rw-rw-rw- 1 vscode vscode 149 Jan 28 08:25 variables.tf

```

6. (Optional) Delete the S3 bucket.

The screenshot shows the AWS Management Console interface for the Amazon S3 service, specifically the 'Buckets' page. The top navigation bar includes the AWS logo, a search bar, and the user's account information (hajra (9416-1806-4465) in the Middle East (UAE) region). The main content area is divided into two tabs: 'General purpose buckets' (selected) and 'Directory buckets'. Under the 'General purpose buckets' tab, there is a section titled 'General purpose buckets (0)' with a sub-header 'Info'. Below this, there are buttons for 'Copy ARN', 'Empty', 'Delete', and 'Create bucket'. A search bar labeled 'Find buckets by name' is present. Below the search bar, a table with columns 'Name', 'AWS Region', and 'Creation date' is shown, but it is empty. A message states 'No buckets. You don't have any buckets.' with a 'Create bucket' button. To the right of the main content area, there are two informational cards: 'Account snapshot' (updated daily) with a 'View dashboard' link, and 'External access summary' (updated daily) with a link to view findings.